



Tecnológico Nacional de México

Instituto Tecnológico Superior de Felipe Carrillo Puerto

Ingeniería en Sistemas Computacionales



Asignatura

Programación web del lado servidor

Tema

2. API's

EVIDENCIA DE APRENDIZAJE

AA 2.1 API Reporte practico

Profesor

MTI. Paloma Gongora Sabino

Alumno (s):

Victor Enrique Tuz Dzidz

ISC-8B

Felipe Carrillo Puerto a 23 de marzo de 2026



2026
año de
**Margarita
Maza**



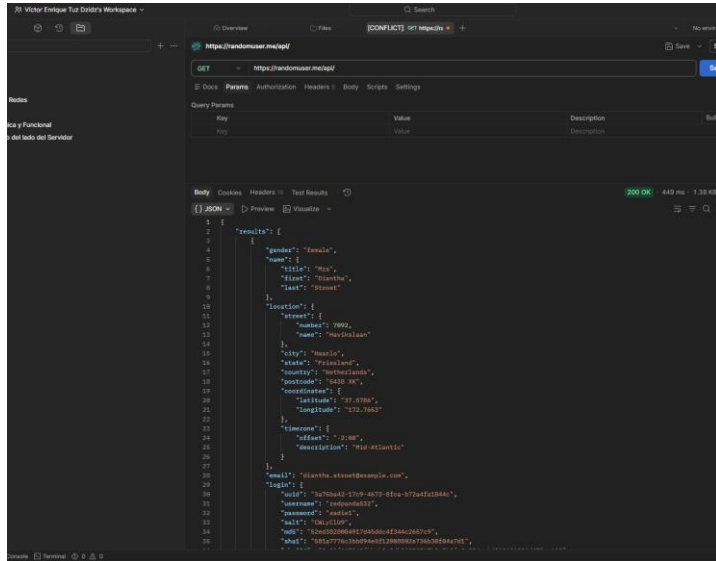


El formato a la peticiones API, requiere comprender los conceptos de:

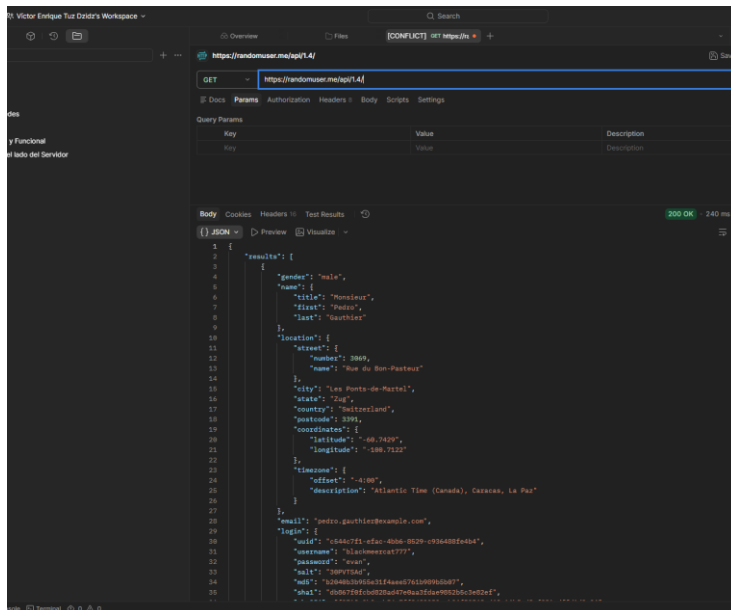
- Endpoints.

URLBase/endpoint

<https://randomuser.me/api/>

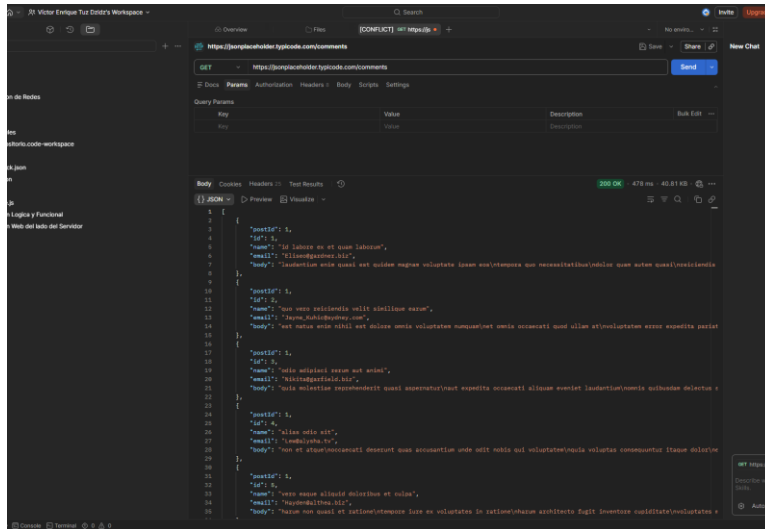


<https://randomuser.me/api/1.4/>





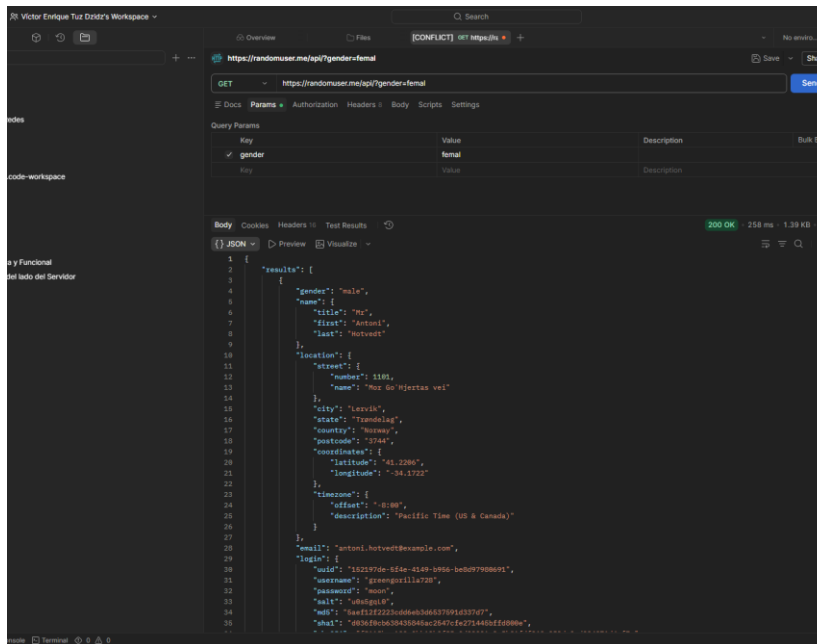
<https://jsonplaceholder.typicode.com/comments>



- Parámetros Query.

URLBase/endpoint/**query**

<https://randomuser.me/api/?gender=female>



2026
año de
Margarita Maza

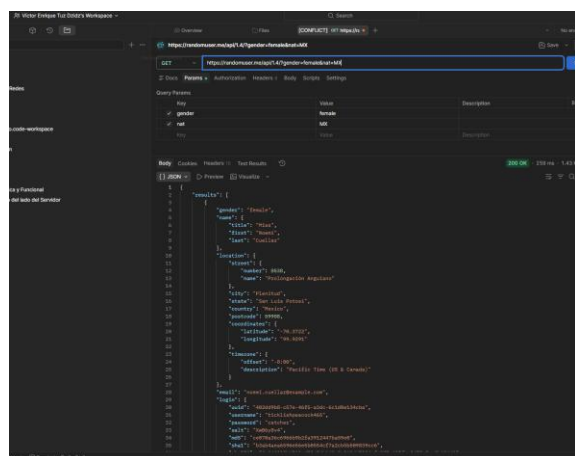




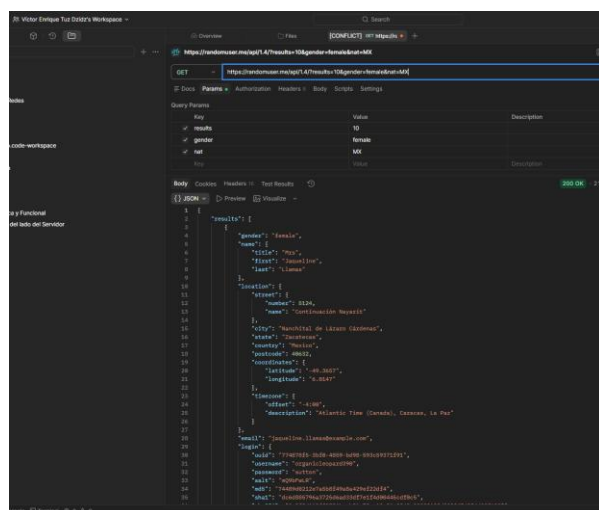
Query se añaden al final de la URL base tras un signo de interrogación (?). El formato es parámetro=valor.

Para especificar si desea que se generen solo usuarios masculinos o femeninos añadiendo el parámetro de género a su solicitud. Los valores válidos para el parámetro de género son "masculino" o "femenino", o puede dejarlo en blanco. Cualquier otro valor hará que el servicio devuelva usuarios masculinos y femeninos.

<https://randomuser.me/api/1.4/?gender=female&nat=MX>



<https://randomuser.me/api/1.4/?results=10&gender=female&nat=MX>



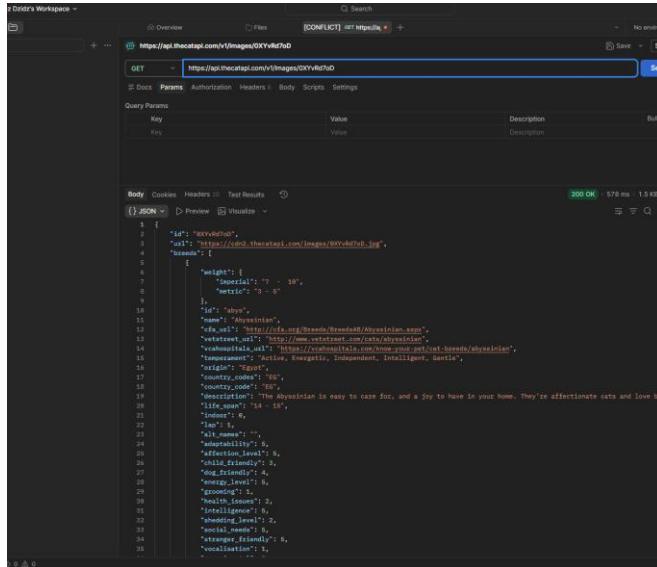


- Parámetros Path

URLBase/endpoint/{**parametro**}

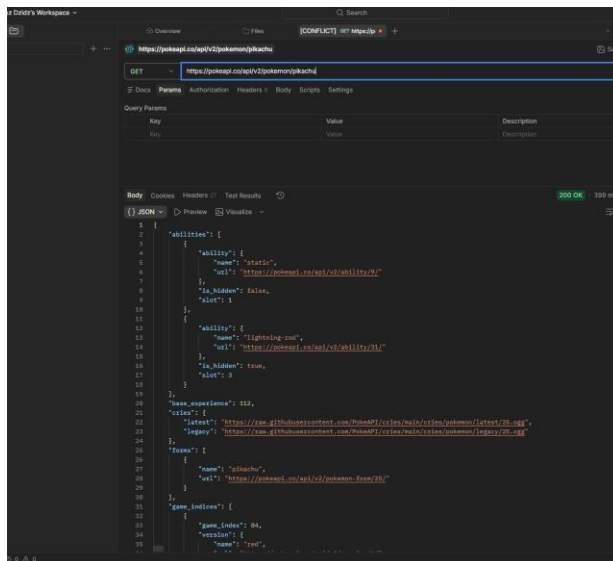
CatAPI

<https://api.thecatapi.com/v1/images/0XYvRd7oD>



PokeAPI

<https://pokeapi.co/api/v2/pokemon/pikachu>



2026
año de
Margarita
Maza





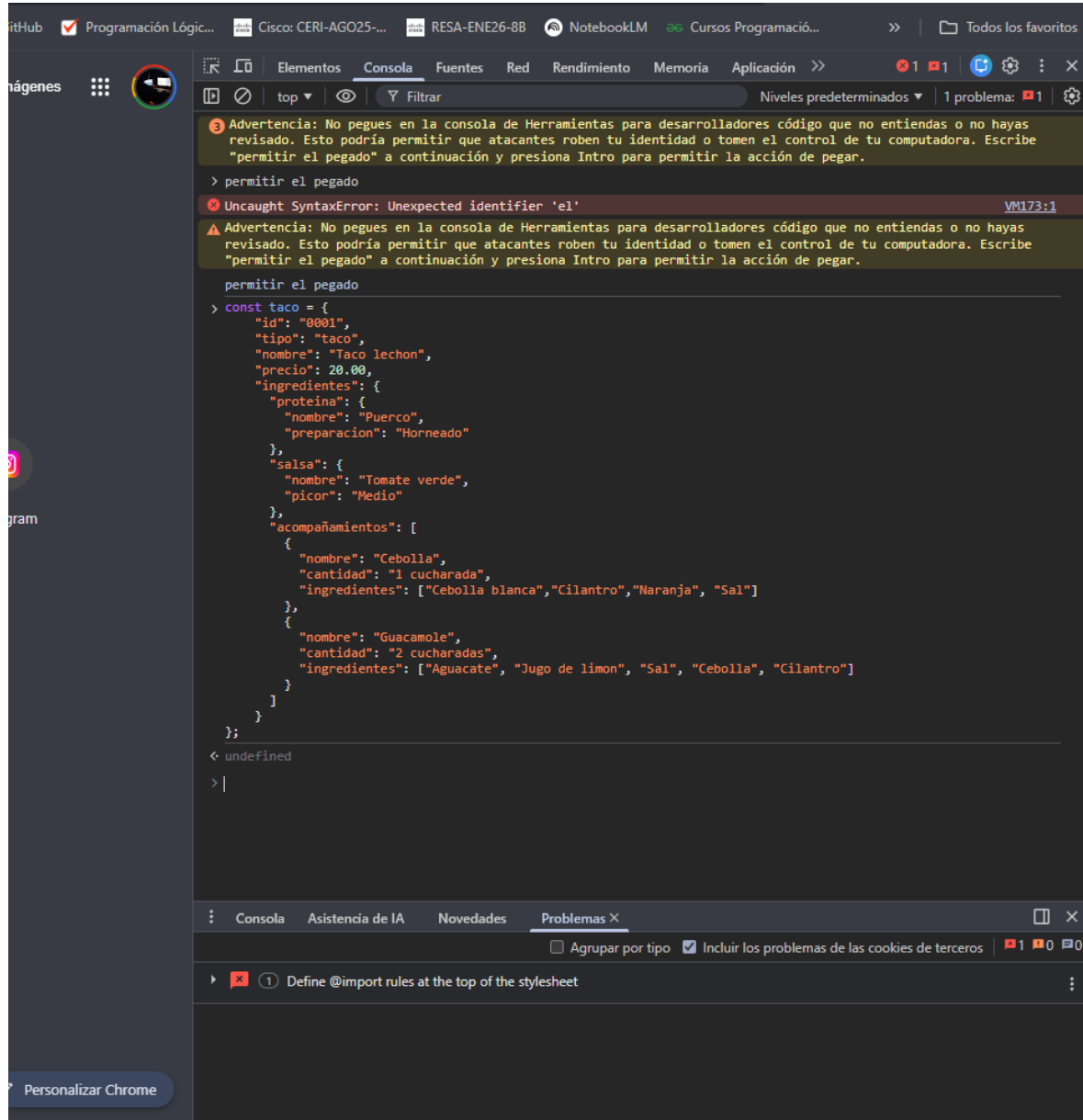
Manipulación de JSON y el DOM

Objetivo: Aprender a navegar por estructuras de datos complejas (objetos anidados y arreglos) utilizando la notación de punto y corchetes.

1. Abre las herramientas de desarrollador en tu navegador (F12 o Ctrl+Shift+I).
2. Ve a la pestaña **Console**.
3. Copia y pega el siguiente objeto **taco** y presiona Enter:

```
const taco = {  
  "id": "0001",  
  "tipo": "taco",  
  "nombre": "Taco lechon",  
  "precio": 20.00,  
  "ingredientes": {  
    "proteina": {  
      "nombre": "Puerco",  
      "preparacion": "Horneado"  
    },  
    "salsa": {  
      "nombre": "Tomate verde",  
      "picor": "Medio"  
    },  
    "acompañamientos": [  
      {  
        "nombre": "Cebolla",  
        "cantidad": "1 cucharada",  
        "ingredientes": ["Cebolla blanca", "Cilantro", "Naranja", "Sal"]  
      },  
      {  
        "nombre": "Guacamole",  
        "cantidad": "2 cucharadas",  
        "ingredientes": ["Aguacate", "Jugo de limon", "Sal", "Cebolla", "Cilantro"]  
      }  
    ]  
  }  
};
```







Escribe el comando `console.log()` necesario para extraer la siguiente información:

4. Obtén el nombre de la proteína ("Puerco").

```
};  
< undefined  
> console.log(taco.ingredientes.proteina.nombre);  
Puerco VM412:1  
< undefined  
> |
```

Consola Asistencia de IA Novedades Problemas X

5. Accede al primer elemento de la lista de ingredientes de la Cebolla ("Cebolla blanca").

```
< undefined  
> console.log(taco.ingredientes.acompañamientos[0].ingredientes[0]);  
Cebolla blanca VM417:1  
< undefined  
> |
```

6. Accede al último elemento de la lista de ingredientes del Guacamole ("Cilantro").

```
> console.log(taco.ingredientes.acompañamientos[1].ingredientes[4]);  
Cilantro VM421:1  
< undefined  
>
```




Renderizar información dinámica desde un objeto JSON hacia el navegador utilizando JavaScript y manipulación del DOM.

1. Crea un archivo llamado index.html y utiliza el siguiente código base. Tu tarea es completar la lógica de JavaScript para que la información del taco aparezca en la tarjeta.

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Receta</title>
  <style>
    .card { border: 2px solid #2ecc71; border-radius: 10px; padding: 20px; width: 300px; font-family: sans-serif; }
    .precio { color: #e67e22; font-weight: bold; font-size: 1.2em; }
    ul { padding-left: 20px; }
  </style>
</head>
<body>

  <div id="taco-card" class="card">
    <h1 id="taco-nombre">Cargando...</h1>
    <p class="precio">Precio: <span id="taco-precio">0</span></p>
    <h3>Acompañamientos:</h3>
    <ul id="lista-acompañamientos">
      </ul>
    </div>

<script>
  const taco = { /* ... (Usar el objeto JSON anterior) ... */ };

  // TU CÓDIGO AQUÍ:
  // 1. Insertar nombre en el h1
  // 2. Insertar precio con formato moneda
  // 3. Recorrer acompañamientos y crear <li> para cada uno
</script>
</body>
</html>
```





Mi código

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Actividad: Renderizado de JSON</title>
  <style>
    body { font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif; display:
flex; justify-content: center; padding: 50px; background-color: #f4f4f4; }
    .card { border: 2px solid #2ecc71; border-radius: 12px; padding: 25px; width:
320px; background-color: white; box-shadow: 0 4px 8px rgba(0,0,0,0.1); }
    h1 { color: #27ae60; margin-top: 0; font-size: 1.8em; }
    .precio { color: #e67e22; font-weight: bold; font-size: 1.3em; margin: 10px 0;
}

    h3 { border-bottom: 1px solid #eee; padding-bottom: 5px; color: #34495e; }
    ul { padding-left: 20px; list-style-type: square; }
    li { margin-bottom: 8px; color: #555; }
  </style>
</head>
<body>

  <div id="taco-card" class="card">
    <h1 id="taco-nombre">Cargando...</h1>
    <p class="precio">Precio: <span id="taco-precio">0</span></p>

    <h3>Acompañamientos:</h3>
    <ul id="lista-acompañamientos">
      </ul>
  </div>

  <script>
    // EL OBJETO JSON (Tus datos)
    const taco = {
      "id": "0001",
      "tipo": "taco",
      "nombre": "Taco lechon",
      "precio": 20.00,
      "ingredientes": {
        "proteina": { "nombre": "Puerco", "preparacion": "Horneado" },
        "salsa": { "nombre": "Tomate verde", "picor": "Medio" },
        "acompañamientos": [
```



2026
año de
Margarita Maza





```
        { "nombre": "Cebolla", "cantidad": "1 cucharada" },
        { "nombre": "Guacamole", "cantidad": "2 cucharadas" }
      ]
    }
  };

  // LÓGICA PARA MANIPULAR EL DOM

  // 1. Insertamos el nombre
  const elNombre = document.getElementById('taco-nombre');
  elNombre.textContent = taco.nombre;

  // 2. Insertamos el precio con formato $
  const elPrecio = document.getElementById('taco-precio');
  elPrecio.textContent = `$$${taco.precio.toFixed(2)}$`;

  // 3. Llenamos la lista de acompañamientos
  const laLista = document.getElementById('lista-acompañamientos');

  taco.ingredientes.acompañamientos.forEach(item => {
    // Creamos el elemento de lista <li>
    const nuevoLi = document.createElement('li');

    // Le asignamos el texto (Nombre + Cantidad)
    nuevoLi.textContent = `${item.nombre}: ${item.cantidad}`;

    // Lo "pegamos" dentro del <ul>
    laLista.appendChild(nuevoLi);
  });
</script>
</body>
</html>
```

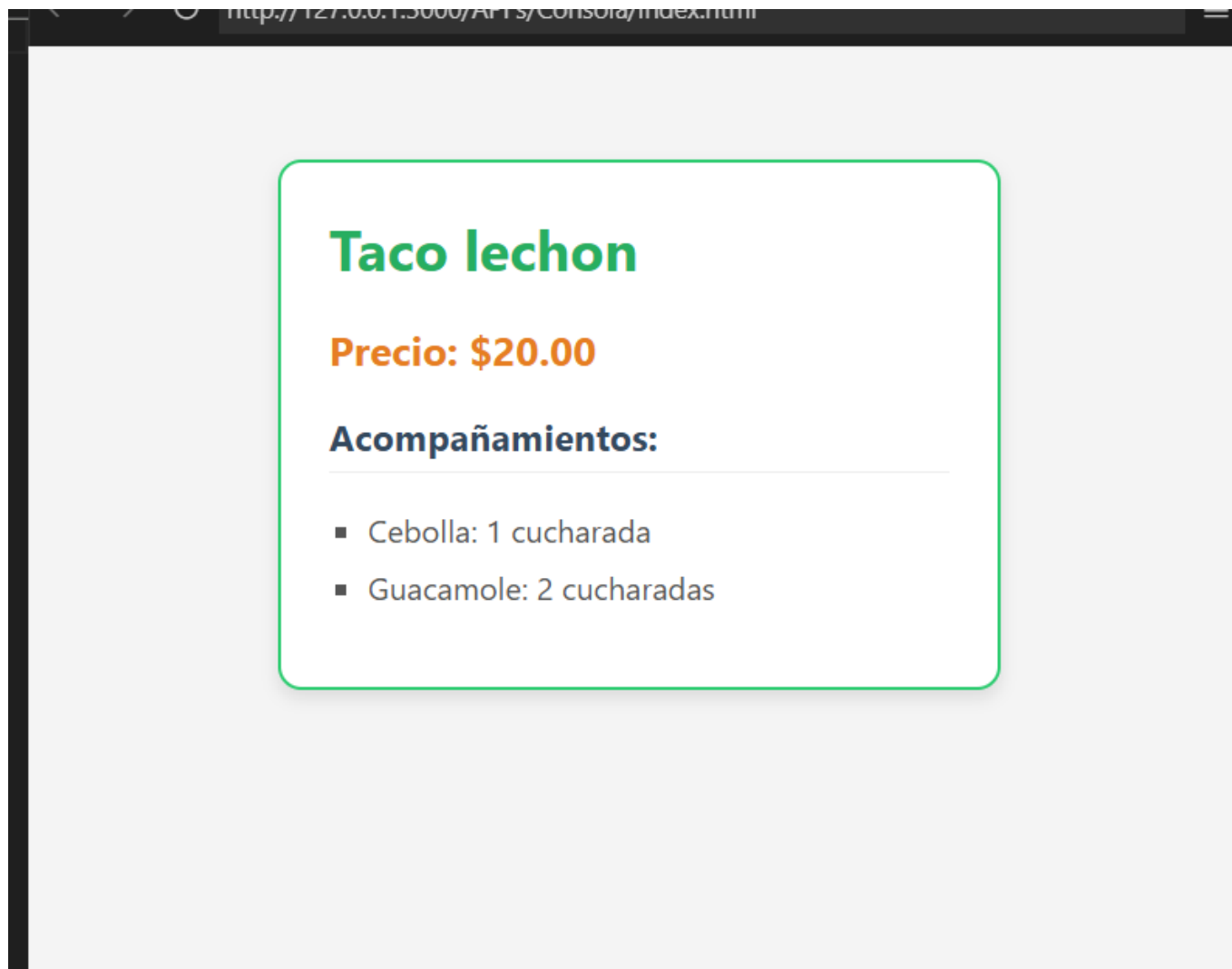


2026
año de
**Margarita
Maza**





Visualización en un navegador





JSON (JavaScript Object Notation)

JSON es un formato ligero para el intercambio de datos, basado en la notación de objetos de JavaScript. Se utiliza ampliamente para estructurar datos que se envían a través de Internet.

Los datos en JSON se organizan en pares de clave-valor y pueden incluir arrays y objetos anidados.

La principal diferencia entre un objeto JavaScript y un JSON es que las claves en JSON deben estar entre comillas.

Objeto en JS

```
const objetoJavaScript = {  
  nombre: "Taco de Pollo",  
  ingredientes: {  
    proteina: "Pollo",  
    salsa: "Salsa Verde"  
  }  
};
```

JSON

```
{  
  "nombre": "Taco de Pollo",  
  "ingredientes": {  
    "proteina": "Pollo",  
    "salsa": "Salsa Verde"  
  }  
}
```

1. Para convertir el objeto en una cadena JSON. Crea un archivo .js en tu editor VSC y copia el código del objeto. Seguidamente copia el siguiente código:

```
// Serializar es convertir a JSON  
const jsonString = JSON.stringify(objetoJavaScript);  
  
console.log(jsonString);
```





2. Observa el resultado.

```
PS C:\Users\tuzdz\Desktop\8B_Visual\Programacion Web del lado del Servidor\API's\Consola> node serializar.js
Resultado de la Serialización:
{"nombre":"Taco de Pollo","ingredientes":{"proteina":"Pollo","salsa":"Salsa Verde"}}
PS C:\Users\tuzdz\Desktop\8B_Visual\Programacion Web del lado del Servidor\API's\Consola> |
```

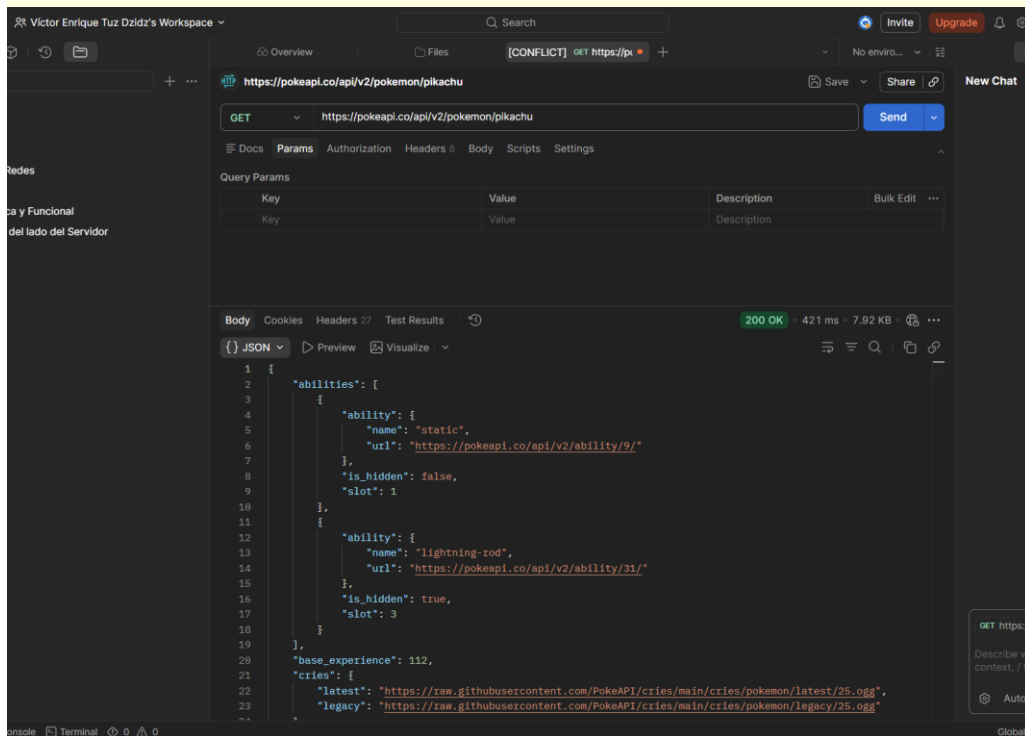
3. Para realizar la conversión contraria de un JSON a un objeto JS. Crea otro archivo y copia el siguiente código:

```
1 // Cadena JSON
2 const jsonString = '{"nombre":"Taco de Pollo","ingredientes":{"proteina":"Pollo",
3 "salsa":"Salsa Verde"}}';
4
5 // Deserializar para convertir a objeto JavaScript
6 const objetoDeserializado = JSON.parse(jsonString);
7
8 console.log(objetoDeserializado);
```

Visualizador de JSON

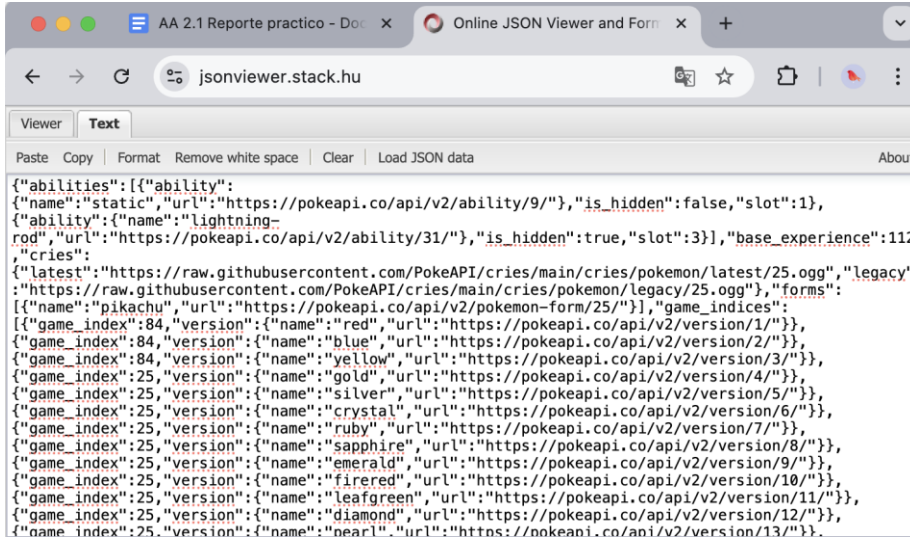
4. Ejecuta la siguiente petición GET en Postman:

<https://pokeapi.co/api/v2/pokemon/pikachu>

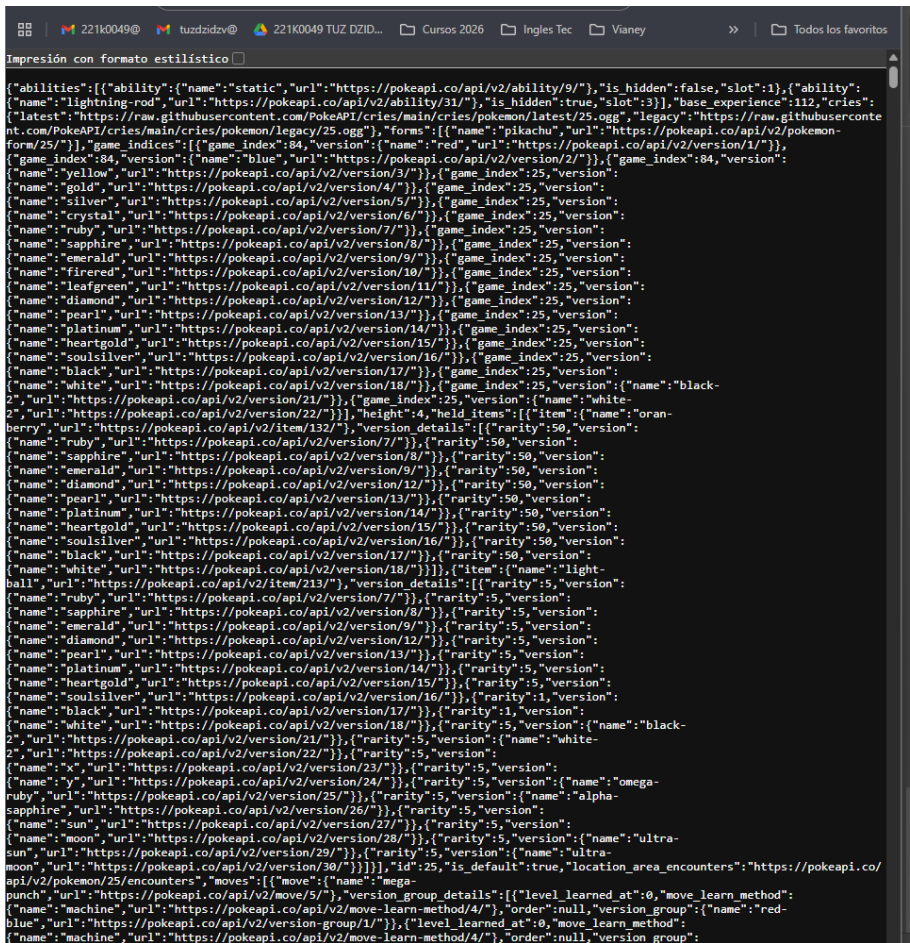




5. Copia el resultado y pega en la pestaña Text en el [visualizador de JSON](#)



Mi Visualización



2026
año de
**Margarita
Maza**





6. Abre la pestaña Viewer e interpreta la información obtenida.

Name	Value
abilities	...
base_experience	112
cries	...
forms	...
game_indices	...
height	4
held_items	...
id	25
is_default	true
location_area_encounters	"https://pokeapi.co/api/v2/encounters"
moves	...
name	"pikachu"
order	35
past_abilities	...
past_types	...
species	...
sprites	...
stats	...
types	...
weight	60

MI visualización

Nombre	Valor
talento	...
experiencia_básica	112
gritos	...
formas	...
índices_de_juego	...
altura	4
objetos_sostenidos	...
id	25
es_predeterminado	verdadero
identificación	25
es_predeterminado	verdadero
encuentros_en_el_área...	"https://pokeapi.co/api/v2/encounters"
movimientos	...
nombre	"pikachu"
orden	35
habilidades_pasadas	...
estadísticas_pasadas	...
tipos_pasados	...
especies	...
sprites	...
estadísticas	...
tipos	...
peso	60

Apoya este proyecto de un solo friki, elimina los anuncios y aumenta la velocidad de carga por solo 19 \$ (pago único): [M](#)





7. Crea un archivo json con el nombre **recetaTacos**. Agrega 4 tacos diferentes típicos de la península, con diferente proteína y acompañamientos. Basate en el siguiente ejemplo:

```
{ } recetaTacos.json > ...
1  [
2    {
3      "id": "0001",
4      "tipo": "taco",
5      "nombre": "Taco lechon",
6      "precio": 20.00,
7      "ingredientes": {
8        "proteina": {
9          "nombre": "Puerco",
10         "preparacion": "Horneado"
11       },
12       "salsa": {
13         "nombre": "Tomate verde",
14         "picor": "Medio"
15       },
16       "acompañamientos": [
17         {
18           "nombre": "Cebolla",
19           "cantidad": "1 cucharada",
20           "ingredientes": ["Cebolla blanca", "Cilantro", "Naranja", "Sal"]
21         },
22         {
23           "nombre": "Guacamole",
24           "cantidad": "2 cucharadas",
25           "ingredientes": ["Aguacate", "Jugo de limon", "Sal", "Cebolla", "Cilantro"]
26         }
27       ]
28     }
29   ]
30
```



Mi código

```

1  {
2    "tipo": "taco",
3    "nombre": "Taco lechon",
4    "precio": 20.00,
5    "ingredientes": {
6      "proteina": { "nombre": "Puerco", "preparacion": "Horneado" },
7      "salsa": { "nombre": "Tomate verde", "picor": "Medio" },
8      "acompañamientos": [
9        { "nombre": "Cebolla", "cantidad": "1 cucharada", "ingredientes": ["Cebolla blanca", "Cilantro"] }
10     ]
11   },
12 },
13 {
14   "id": "0002",
15   "tipo": "taco",
16   "nombre": "Taco Cochinita",
17   "precio": 18.00,
18   "ingredientes": {
19     "proteina": { "nombre": "Cerdo", "preparacion": "Pibil (Achiote)" },
20     "salsa": { "nombre": "Xnipec", "picor": "Alto" },
21     "acompañamientos": [
22       { "nombre": "Cebolla Morada", "cantidad": "2 cucharadas", "ingredientes": ["Cebolla morada", "Naranja agria"] }
23     ]
24   },
25 },
26 {
27   "id": "0003",
28   "tipo": "taco",
29   "nombre": "Taco Relleno Negro",
30   "precio": 22.00,
31   "ingredientes": {
32     "proteina": { "nombre": "Pavo", "preparacion": "Guisado en recado negro" },
33     "salsa": { "nombre": "Habanero", "picor": "Muy Alto" },
34     "acompañamientos": [
35       { "nombre": "Huevo cocido", "cantidad": "1 pieza", "ingredientes": ["Huevo", "Sal"] }
36     ]
37   },
38 },
39 {
40   "id": "0004",
41   "tipo": "taco",
42   "nombre": "Taco Poc Chuc",
43   "precio": 25.00,
44   "ingredientes": {
45     "proteina": { "nombre": "Cerdo", "preparacion": "Asado al carbón" },
46     "salsa": { "nombre": "Chiltomate", "picor": "Bajo" },
47     "acompañamientos": [
48       { "nombre": "Frijoles", "cantidad": "3 cucharadas", "ingredientes": ["Frijol"]
49     ]
50   },
51 },
52 ]

```





Aplicación usando datos JSON y API Fetch.

1. Crea el proyecto con el nombre **AppRecetaTacos**
2. Crea el archivo **index.js**
3. Agregar el archivo **recetaTacos.json**
4. Inicializa npm
5. Agrega el tipo **module** como sistema de módulos en el archivo configuración.
6. Instala el framework **express** y el middleware **body-parser**
7. Crea el boilerplate de la aplicación:
 - a. Importar las dependencias instaladas.
 - b. Crear la instancia de express
 - c. Configurar el puerto
 - d. Configurar el método para iniciar el servidor.
 - e. Probar funcionalidad
8. Crear el directorio **public** con los archivos **index.html** y **estilo.css**
9. Crea el boilerplate para un documento html.
 - a. DOCTYPE y HTML.
 - b. Cabecera <head>, para definir título para la página TacosMx
 - c. Enlazar con la [hoja de estilos](#) que se utilizará para la página.
 - d. Integrar al cuerpo <body>, el título principal (<h1>Tacoschidos 🌮 </h1>).
 - e. Incluye en el body un formulario con varios botones, cada botón representa la proteína de cada tipo de taco. Finalmente deben haber 4 botones. Todos los botones tendrán un atributo data-type que se utiliza para identificar el tipo de taco seleccionado. La estructura es la siguiente:

```
<form id="Form" class="buttons">
  <button type="button" data-type="Puerco">🐷</button>
</form>
```

- f. Después del formulario, integrar el siguiente elemento contenedor:





```
<div id="Container"></div>
```

- g. Para obtener los datos del backend y renderizarlos en el navegador utilizando el DOM de JS, se integrará el [script](#) utilizando la API Fetch.

10. Desarrolla el backend:

- Crea una constante **recetaJSON** y asigna entre backticks la cadena JSON generada en tu archivo `recetaTacos.json`.
- Deserializa (convierte a objeto JS) la cadena JSON y asigna a una constante **recetasTacos**.
- Utiliza el middleware **express.static** para servir los archivos `index.html` y `estilo.css` desde la carpeta `public`:

```
nombre_servidor.use(express.static("public"));
```

- Utiliza el middleware **bodyParser.json()** para acceder a los datos enviados en el cuerpo de una solicitud.
- Crea el handler **get** para obtener la receta de un taco en específico. El endpoint es **/receta/:type**. El objetivo del siguiente código es encontrar una receta de taco que coincida con el tipo de proteína especificado en la solicitud. El contenido del handler debe ser el siguiente:

```
const elegirTaco = recetasTacos.find(r => r.ingredientes.proteina.nombre.toLowerCase() === req.params.type.toLowerCase());  
res.json(elegirTaco || { error: "Receta no encontrada" });
```

`req.params.type()` proviene de la ruta dinámica en la URL, como `/receta/:type`, donde `:type` es un valor proporcionado por el cliente ("pollo" o "puerco").

Código HTML

```
<!DOCTYPE html>  
<html lang="es">  
<head>  
  <meta charset="UTF-8">  
  <title>TacosMx</title> <link rel="stylesheet" href="estilo.css"> </head>  
<body>
```





```
<h1>Tacoschidos</h1> <form id="Form" class="buttons">
  <button type="button" data-type="Puerco">🐖</button>
  <button type="button" data-type="Pollo">🐔</button>
  <button type="button" data-type="Pavo">🦃</button>
  <button type="button" data-type="Res">🐄</button>
</form>

<div id="Container"></div>

<script>
  document.querySelectorAll("#Form button").forEach(button => {
    button.addEventListener("click", async (event) => {
      const type = event.target.getAttribute("data-type").toLowerCase();

      const response = await fetch(`/receta/${type}`);
      const receta = await response.json();

      const contenedor = document.getElementById("Container");

      if (receta.error) {
        contenedor.innerHTML = "<h2>Receta no encontrada</h2>";
      } else {
        contenedor.innerHTML = `
          <h2>${receta.nombre}</h2>
          <h3>Ingredientes:</h3>
          <p>Carne de: ${receta.ingredientes.proteina.nombre}, Modo
preparación: ${receta.ingredientes.proteina.preparacion}</p>
          <p>Salsa: ${receta.ingredientes.salsa.nombre}
(${receta.ingredientes.salsa.picor})</p>
          ${receta.ingredientes.acompañamientos.map(a =>
            `<p>Acompañamiento: ${a.cantidad} : ${a.nombre}</p>`
          ).join("")}`;
      }
    });
  });
</script>
</body>
</html>
```





```
html {  
  height: 100%;  
}  
  
body {  
  font-family: Arial, sans-serif;  
  margin: 20px;  
  padding: 0;  
  text-align: center;  
}  
  
h1 {  
  display: inline;  
  font-size: 5rem;  
  text-align: center;  
  color: #e26e5c;  
  background-color: white;  
}  
  
h2 {  
  color: #195784;  
}  
  
.buttons {  
  display: flex;  
  justify-content: center;  
  margin: 20px;  
}  
  
button {  
  font-size: 5rem;  
}  
  
.buttons button {  
  display: flex;  
  flex-direction: column;  
  align-items: center;  
  justify-content: center;  
  border: none;  
  background-color: white;  
  cursor: pointer;  
  padding: 0;
```



2026
año de
**Margarita
Maza**





```
margin: 0 10px;
width: 180px;
height: 180px;
border-radius: 50%;
box-shadow: 0 2px 5px rgba(0, 0, 0, 0.2);
transition: transform 0.3s, box-shadow 0.3s;
}

.buttons button:hover {
  transform: scale(1.05);
  box-shadow: 0 4px 10px rgba(0, 0, 0, 0.2);
}

.buttons button:focus {
  outline: 2px solid #f7fa5858;;
}

#Container {
  background-color: white;
  margin-top: 40px;
  display: flex;
  flex-direction: column;
  align-items: center;
}

#ingredientesLista {
  margin: 0;
  padding: 0;
  list-style-type: none;
}

#ingredientesLista li {
  margin-bottom: 10px;
  color: #555;
}
```



2026
año de
Margarita Maza





Código JS

```
import express from "express"; // [cite: 187]
import bodyParser from "body-parser"; // [cite: 187]
const app = express(); // [cite: 188]
const port = 3000; // [cite: 189]
// Configuración de middlewares [cite: 211, 213]
app.use(express.static("public"));
app.use(bodyParser.json());

// 10.a. Cadena JSON (Serializada) [cite: 209]
// REVISAR: Asegúrate de que termine con `]; al final.
const recetaJSON = `[
  {
    "id": "0001",
    "nombre": "Taco lechon",
    "ingredientes": {
      "proteina": { "nombre": "Puerco", "preparacion": "Horneado" },
      "salsa": { "nombre": "Tomate verde", "picor": "Medio" },
      "acompañamientos": [
        { "nombre": "Cebolla", "cantidad": "1 cucharada" },
        { "nombre": "Guacamole", "cantidad": "2 cucharadas" }
      ]
    }
  }
]`;

// 10.b. Deserialización (Convertir a objeto JS)
const recetasTacos = JSON.parse(recetaJSON);

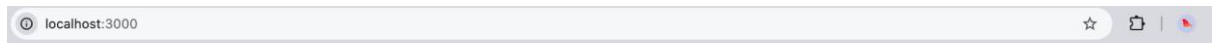
// 10.e. Handler GET para obtener la receta
app.get("/receta/:type", (req, res) => {
  // Buscamos el taco que coincida con el tipo [cite: 215, 218]
  const elegirTaco = recetasTacos.find(r =>
    r.ingredientes.proteina.nombre.toLowerCase() === req.params.type.toLowerCase()
  );
  // Enviamos el JSON encontrado o el error
  res.json(elegirTaco || { error: "Receta no encontrada" });
});

// 7.d. Iniciar el servidor
app.listen(port, () => {
  console.log(`Servidor iniciado en http://localhost:${port}`);
});
```





f. Prueba la funcionalidad. El resultado deberá ser similar a la imagen.



Tacoschidos 🌮



Taco lechon

Ingredientes:

Carne de: Puerco, Modo preparación: Horneado

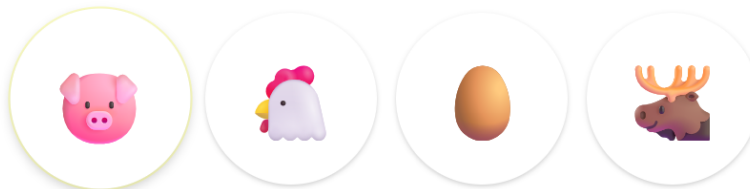
Salsa: Tomate verde (Medio)

Acompañamiento: 1 cucharada : Cebolla

Acompañamiento: 2 cucharadas : Guacamole

Resultado de la practica

Tacoschidos



Taco lechon

Ingredientes:

Carne de: Puerco, Modo preparación: Horneado

Salsa: Tomate verde (Medio)

Acompañamiento: 1 cucharada : Cebolla

Acompañamiento: 2 cucharadas : Guacamole



2026
año de
**Margarita
Maza**

